# Design Document: Multi-Threaded Math Server

**By: Nguyen Do (npd220001) and Jeremiah Boban (jxb220076)**

1. **Project Overview**
    - This document covers the design and implementation of a centralized Math Server and the corresponding client application. The system is built using TCP Sockets in Java, and follows a standard client-server communication pattern.

2. **System architecture**
    - Networking model: This network application uses a client-server architecture
    - Concurrency model: For every client connecting to the network, the server treats them as a separate thread. It uses a server socket to look for incoming connections. Any accepted connections will create a Producer-Consumer pattern, where the ClientHandler object puts requests into a queue, and the RequestProcessor object takes them out to resolve. This approach allows the server to be responsive to all other clients. The server also treats every request as an object, by placing them in a queue, it ensures that requests are processed in the order they enter the buffer.
    - Transport layer: TCP is used for its reliability, and it can preserve the order of delivered math equations and results to each client.

3. **System algorithm**
    - Shunting yard algorithm: This tool takes broken down equation strings, and by treating them as postfix expressions, it can determine the operator precedence and produce accurate results.
    - Algorithm Logic:
        - Infix to Postfix: The algorithm uses an operator stack and an output queue to create a postfix expression. A separate operand stack is used to evaluate expressions and create the results. These stacks are unique to each client, but the server will handle all stacks as it completes the client's requests.
        - Precedence: Operators have precedence value based on PEMDAS to ensure mathematical correctness (Ex: + and - = 1, * and / = 2, etc.)
        - Thread Safety: Each thread has its local stacks within the solveEquation method, preventing corrupted data or thread conflicts within these stacks during simultaneous client connections. A queue is added for requests to further prevent thread conflicts. This way, the server can process each request even if clients send theirs simultaneously.

4. **Protocol Design Specification:**
    - Each client when connected to the server has its own socket. The queue is the buffer where requests are waiting to be processed. The processor is the main function of the server, solving equations one-by-one.
    - Message format for sending and receiving math calculations:

        - Request: Encoded string containing operators and operands, separated by a single space (Ex: 1 + 4 * 12 \n), where \n is caused by pressing the Enter key, which tells the server that the message is completed.
        - Response: Encoded string with unique status prefix.
            - Success: Result: <value> \n (Ex: Result: 49 \n)
            - Failure: Error: <description> \n (Ex: Error: Unknown token ‘a’ \n)
    - Message format for joining and terminating the connection.
        - Joining: The client types in their name as a string, followed by \n.
            - The server responds with an ACK string: Welcome client <name>, you joined at time: yyyy/MM/dd HH:mm:ss \n
        - Terminating: When the client encounters the message: Do you like to stay for 3 more equations? (YES/NO), they can type NO to terminate the connection.
            - The client will see the message: “Connection closed at yyyy/MM/dd HH:mm:ss. Duration: <value> s"
    - Format for keeping logs of clients’ activities on the server side
        - The server uses a log format to keep track of client’s activities and requests
            - Connection: Client <name> connected at yyyy/MM/dd HH:mm:ss”
            - Request: Request from client <name>: <equation>
            - Disconnection: Client <name> disconnected at yyyy/MM/dd HH:mm:ss. Duration <value>s
        - In a separate log file, the server can also keep track of client activities in a formal manner:
            - Connection: [yyyy/MM/dd HH:mm:ss] JOIN: <client name>
            - Request: [yyyy/MM/dd HH:mm:ss] REQUEST from <client name>: <request>]
            - Disconnection: [yyyy/MM/dd HH:mm:ss] QUIT: <client name>. Duration: <value> s

5. **Data Management and Logging**
    - Connection and termination times: The server uses java.time.LocalDateTime to timestamp client join and leave times in a connection.
    - Session times: The server records each client’s connection duration using System.currentTimeMillis() to calculate the total session duration (in seconds)

6. **Assumptions**
    - Equation format: Users must separate every token (operators, numbers, parentheses) with a space, which allows the parser to work correctly.
    - Port Availability: The system assumes Port 6789 to be opened and not blocked by a firewall on the host machine. Hence, this port was hardcoded in the program. The IP Address 127.0.0.1 was also hardcoded as the local machine to compile the program.